



## Inhaltsverzeichnis

|                                                           |    |
|-----------------------------------------------------------|----|
| Widmung .....                                             | 5  |
| Kapitel 1: Einführung ins Programmieren.....              | 6  |
| Erklärung: Was ist Programmieren? .....                   | 6  |
| Erklärung: Was ist C#? .....                              | 7  |
| Warum C# cool ist:.....                                   | 7  |
| Visualisierung: Dein erster Codi-Moment .....             | 7  |
| Code: Dein erstes kleines Programm.....                   | 8  |
| Aufgabe: Mach Codi persönlich.....                        | 8  |
| ☆☆ Bonus: Codi kann lachen 😊 .....                        | 9  |
| Lösung:.....                                              | 9  |
| Kapitel 2: Erste Schritte.....                            | 10 |
| Erklärung: Dein erstes richtiges Programm starten.....    | 10 |
| Visualisierung: Startsignal für Codi.....                 | 11 |
| Code: „Hallo Codi“ erweitern.....                         | 11 |
| Aufgabe: Begrüße Codi auf neue Weise.....                 | 12 |
| ☆☆ Bonus: ASCII-Kunst einfügen .....                      | 12 |
| Lösung:.....                                              | 13 |
| Kapitel 3: Grundlagen .....                               | 14 |
| Erklärung: Variablen – Codis Kisten .....                 | 14 |
| Visualisierung: Variablen-Kisten .....                    | 14 |
| Erklärung: Datentypen – was in die Kiste passt .....      | 15 |
| Erklärung: if/else – Entscheidungen treffen.....          | 15 |
| Erklärung: Schleifen – Aufgaben wiederholen .....         | 16 |
| Aufgabe: Dein Codi-Abenteuer.....                         | 17 |
| ☆☆ Bonus: Zufallsgenerator für Überraschungen .....       | 18 |
| Lösung:.....                                              | 18 |
| Kapitel 4: Methoden – eigene kleine Programme bauen ..... | 19 |
| Erklärung: Methoden – Codis Superkräfte.....              | 19 |
| Visualisierung: Methoden als Schubladen.....              | 19 |
| Code: Eine Methode erstellen.....                         | 20 |
| Erklärung: Methoden mit Eingaben (Parameter).....         | 20 |
| Erklärung: Methoden mit Rückgabewert (return) .....       | 21 |
| Aufgabe: Codi als Superkoch .....                         | 22 |

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| ★★★ Bonus: Methoden kombinieren .....                                   | 22 |
| Kapitel 5: Objektorientierung – Klassen & Objekte (Katze-Version) ..... | 24 |
| Erklärung: Klassen – Codis Baupläne .....                               | 24 |
| Visualisierung: Klasse vs. Objekt .....                                 | 25 |
| Code: Klasse und Objekt.....                                            | 25 |
| Erklärung: Konstruktoren – Objekte sofort starten .....                 | 26 |
| Erklärung: Mehrere Objekte – viele Katzen .....                         | 27 |
| Aufgabe: Deine eigene Katze.....                                        | 27 |
| ★★★ Bonus: Methoden mit Parameter .....                                 | 27 |
| Lösung: Katzen-Klasse.....                                              | 28 |
| Kapitel 6: Mini-Projekte .....                                          | 29 |
| Erklärung: Jetzt wird gebaut! .....                                     | 29 |
| Projekt 1: Zahlenraten-Spiel .....                                      | 30 |
| Erklärung .....                                                         | 30 |
| Visualisierung.....                                                     | 30 |
| Code: .....                                                             | 31 |
| Aufgabe: .....                                                          | 31 |
| ★★★ Bonus: Füge einen Versuchszähler hinzu! .....                       | 31 |
| Lösung (mit Zähler): .....                                              | 32 |
| Projekt 2: Quiz-Spiel .....                                             | 32 |
| Erklärung .....                                                         | 32 |
| Visualisierung.....                                                     | 32 |
| Code: .....                                                             | 33 |
| Aufgabe: .....                                                          | 33 |
| ★★★ Bonus: Mische Fragen zufällig mit Random.....                       | 33 |
| Projekt 3: Mini-Abenteuer .....                                         | 34 |
| Erklärung .....                                                         | 34 |
| Visualisierung: .....                                                   | 34 |
| Code: .....                                                             | 35 |
| Aufgabe: .....                                                          | 35 |
| ★★★ Bonus: Nutze Methoden für einzelne Szenen .....                     | 35 |
| Lösung (Idee): .....                                                    | 36 |
| Kapitel-Zusammenfassung .....                                           | 36 |

|                                                                                                          |    |
|----------------------------------------------------------------------------------------------------------|----|
| NÄCHSTER SCHRITT: FINALES PROJEKT.....                                                                   | 36 |
|  Snake-Spiel in C#..... | 37 |
| Kapitel 7: Das Snake-Spiel programmieren.....                                                            | 37 |
| Erklärung: Was ist Snake? .....                                                                          | 37 |
| Visualisierung: .....                                                                                    | 38 |
| Schritt 1: Spielfeld erstellen .....                                                                     | 38 |
| Erklärung .....                                                                                          | 38 |
| Code:.....                                                                                               | 38 |
| Aufgabe: .....                                                                                           | 39 |
| Lösung:.....                                                                                             | 39 |
| Schritt 2: Die Schlange.....                                                                             | 39 |
| Erklärung .....                                                                                          | 39 |
| Code:.....                                                                                               | 39 |
| Aufgabe: .....                                                                                           | 40 |
| Schritt 3: Steuerung .....                                                                               | 40 |
| Erklärung .....                                                                                          | 40 |
| Code:.....                                                                                               | 40 |
| Aufgabe: .....                                                                                           | 40 |
| Schritt 4: Essen .....                                                                                   | 41 |
| Erklärung .....                                                                                          | 41 |
| Code:.....                                                                                               | 41 |
| Aufgabe: .....                                                                                           | 41 |
| Schritt 5: Wachstum.....                                                                                 | 41 |
| Erklärung .....                                                                                          | 41 |
| Code:.....                                                                                               | 41 |
| Aufgabe: .....                                                                                           | 42 |
| Schritt 6: Game Over.....                                                                                | 42 |
| Erklärung .....                                                                                          | 42 |
| Code:.....                                                                                               | 42 |
| Deine Mission.....                                                                                       | 44 |
| Abschluss  .....      | 44 |
| Anhang: So startest du das C#-Programm Snake-Spiel .....                                                 | 45 |
| 1. Code von GitHub herunterladen .....                                                                   | 45 |
| 2. Visual Studio kostenlos installieren .....                                                            | 45 |
| 3. Projekt öffnen und starten .....                                                                      | 48 |

# Widmung

Für meinen lieben Sohn Moaad,

du hast schon früh angefangen, die Welt des Programmierens zu entdecken. Mit deinen ersten Spielen in Scratch hast du gezeigt, wie viel Kreativität, Neugier und Ausdauer in dir steckt.

Dieses Buch ist für dich – damit du deinen Weg als Programmierer weitergehen kannst.  
Ich bin unglaublich stolz auf dich und darauf, wie begeistert du neue Dinge lernst.

Bleib neugierig, probiere Neues aus und hab keine Angst vor Fehlern – genau so entstehen die besten Ideen.

Vielleicht programmierst du eines Tages Spiele oder Programme, die andere Menschen begeistern.

Ich glaube an dich! 

**Dein Papa**

# Kapitel 1: Einführung ins Programmieren

## Erklärung: Was ist Programmieren?

Hey du! 🙌 Stell dir vor, du hast einen kleinen Roboterfreund namens **Codi**. Codi kann alles machen, was du ihm sagst – **aber nur, wenn du die richtigen Anweisungen gibst**.

Programmieren ist genau das: **Du sagst dem Computer Schritt für Schritt, was er tun soll.**

- Eine Anweisung = ein Befehl an Codi
- Ein Programm = eine ganze Reihe von Befehlen, damit Codi etwas Cooles macht

Beispiel: Du willst, dass Codi dir deinen Lieblings-Snack holt.

- Schritt 1: Codi, gehe in die Küche
- Schritt 2: Öffne den Kühlschrank
- Schritt 3: Nimm den Snack
- Schritt 4: Bring ihn zu mir

Siehst du? Jede kleine Anweisung ist wichtig.



## Erklärung: Was ist C#?

C# (gesprochen: „C-Sharp“) ist **eine Programmiersprache**, also die Sprache, mit der du Codi sagst, was er tun soll.

## Warum C# cool ist:

- Einfach zu lernen für Anfänger
- Super für Spiele, kleine Programme und Apps
- Wird von echten Profis verwendet

Wenn C# eine Farbe wäre, wäre es **leuchtend grün** – fröhlich, klar und voller Möglichkeiten.



## Visualisierung: Dein erster Codi-Moment

```
(\_/)\n(  ••  )\n / > 🍪 Codi bringt dir einen Cookie!
```

Bildidee: Ein kleiner Roboter mit einem Tablett, auf dem ein Keks liegt.

## Code: Dein erstes kleines Programm

Wir starten mit dem Klassiker: **Hallo Welt**.

C# sagt damit einfach „Hallo“ auf dem Bildschirm.

```
using System; // Damit wir Computer-Sachen wie Text anzeigen können
```

```
0 references
class Program
{
    0 references
    static void Main()
    {
        Console.WriteLine("Hallo Welt!"); // Codi sagt Hallo
    }
}
```

- `using System;` → Wir nutzen C#-Befehle für Text, Zahlen usw.
- `class Program` → Das ist unser Bauplan für das Programm
- `static void Main()` → Der Startpunkt: Hier beginnt Codi zu arbeiten
- `Console.WriteLine("Hallo Welt!");` → Codi schreibt etwas auf den Bildschirm

Wenn du das Programm startest, siehst du:

```
Hallo Welt!
```

Hurra! 🎉 Dein erstes Programm läuft!

---

## Aufgabe: Mach Codi persönlich

1. Ändere den Text "Hallo Welt!" zu "Hallo [dein Name]!"
2. Lass Codi noch eine zweite Zeile sagen, z. B. "Ich bin dein Freund Codi!"

## ☆☆ Bonus: Codi kann lachen 😄

```
Console.WriteLine("Haha, Codi lacht!");
```

Probiere es aus und beobachte, wie dein Programm lebendig wird.

---

### Lösung:

```
using System;

0 references
class Program
{
    0 references
    static void Main()
    {
        Console.WriteLine("Hallo Alex!"); // Dein Name
        Console.WriteLine("Ich bin dein Freund Codi!"); // Codi stellt sich vor
        Console.WriteLine("Haha, Codi lacht!"); // Codi lacht
    }
}
```

Wenn du das startest, erscheint:

```
Hallo Alex!
Ich bin dein Freund Codi!
Haha, Codi lacht!
```

👀 Super, jetzt weißt du: Programmieren macht Spaß, und Codi kann dir zuhören!

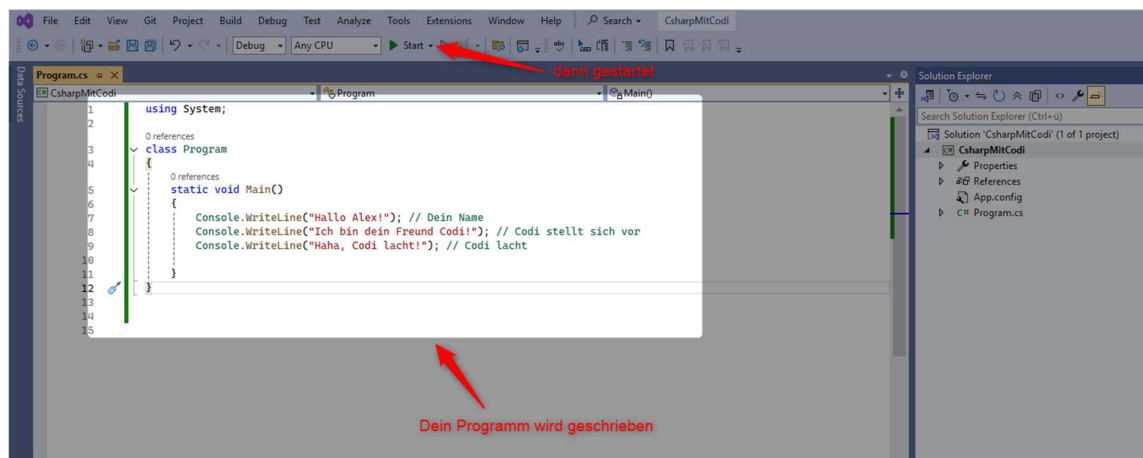
# Kapitel 2: Erste Schritte

## Erklärung: Dein erstes richtiges Programm starten

Jetzt weißt du schon, wie man Codi etwas sagen lässt.

Aber bevor Codi wirklich loslegt, musst du **wissen, wie man ein Programm startet**.

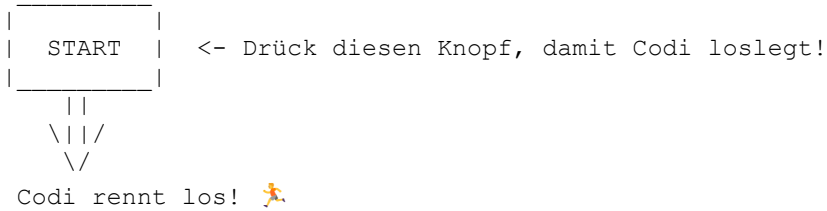
1. Du brauchst einen **Computer**.
2. Du brauchst eine **Programmierungsumgebung** – wir nehmen **Visual Studio** (kostenlos) oder **Visual Studio Code**.
3. Dein Programm wird **geschrieben**, dann **gestartet**, und Codi zeigt dir das Ergebnis.



💡 Vergleich: Programmieren ist wie Kochen:

- Zutaten = Code
- Rezept = Programm
- Herd = Computer
- Fertiges Gericht = das, was auf dem Bildschirm erscheint

## Visualisierung: Startsignal für Codi



Bildidee: Ein bunter Knopf mit „START“, der gedrückt wird und ein kleiner Roboter rennt los.

---

## Code: „Hallo Codi“ erweitern

Wir machen dein erstes richtiges Programm – mit mehreren Zeilen Text.

```
using System; // Damit wir Text anzeigen können

0 references
class Program
{
    0 references
    static void Main()
    {
        Console.WriteLine("Hallo Moaad!"); // Codi begrüßt dich
        Console.WriteLine("Bereit, Programmieren zu lernen?"); // Kleine Frage an dich
        Console.WriteLine("Los geht's! 🚀"); // Motivation
    }
}
```

- Jede `Console.WriteLine` zeigt Text auf dem Bildschirm.
- Du kannst beliebig viele Zeilen hinzufügen.

Wenn du startest, sieht es so aus:

```
Hallo Moaad!
Bereit, Programmieren zu lernen?
Los geht's! 🚀
```

## Aufgabe: Begrüße Codi auf neue Weise

1. Ändere den Namen zu deinem Namen
  2. Füge eine lustige Nachricht hinzu, z. B. "Codi, lass uns ein Spiel machen!"
  3. Experimentiere mit mindestens **5 Zeilen Text**
- 

## ★ ★ Bonus: ASCII-Kunst einfügen

Codi liebt kleine Zeichnungen. Probiere das:

```
Console.WriteLine("  (\\_/_/");  
Console.WriteLine("  (•_•)");  
Console.WriteLine(" / > 🍪 Codi bringt einen Keks!");
```

Das ergibt:

```
(\\_/_/  
(•_•)  
/ > 🍪 Codi bringt einen Keks!
```

## Lösung:

```
using System;
```

0 references

```
class Program
```

```
{
```

0 references

```
static void Main()
```

```
{
```

```
    Console.WriteLine("Hallo Mia!"); // Dein Name
```

```
    Console.WriteLine("Codi, lass uns ein Spiel machen!");
```

```
    Console.WriteLine("Bist du bereit? 🤖");
```

```
    Console.WriteLine("Los geht's! 🚀");
```

```
    Console.WriteLine("Viel Spaß beim Programmieren! 🎮");
```

```
    // ASCII-Kunst
```

```
    Console.WriteLine("  (\\_/)");
```

```
    Console.WriteLine("  (•_•)");
```

```
    Console.WriteLine("  / >🤖 Codi bringt einen Keks!");
```

```
}
```

```
}
```

- 🕒 Wenn du das startest, siehst du eine Begrüßung + kleine Zeichnung – **dein Programm lebt!**

Wir haben jetzt:

- Dein erstes Programm geschrieben
- Es gestartet
- Text und kleine Kunst ausgegeben

Im nächsten Kapitel **lernen wir Variablen und Datentypen kennen**, damit Codi **Dinge speichern und sich merken** kann.

# Kapitel 3: Grundlagen

## Erklärung: Variablen – Codis Kisten

Stell dir vor, Codi hat viele **Kisten**. 📦

In jede Kiste kann er **etwas hineinlegen**, zum Beispiel Zahlen, Wörter oder kleine Infos.

- Jede Kiste hat einen **Namen**, damit Codi weiß, was drin ist.
- Eine Kiste = **Variable**

### Beispiel:

```
int    alter = 12;      // Codi merkt sich, dass du 12 Jahre alt bist
string name  = "Alex";  // Codi merkt sich deinen Namen
```

- `int` = ganze Zahl (z. B. 12, 42, 100)
- `string` = Text (z. B. „Hallo“, „Alex“)

💡 Merke: **Variablen sind wie Kisten mit Namen.**

---

## Visualisierung: Variablen-Kisten

```
+-----+
| Name: Alex |
+-----+

+-----+
| Alter: 12  |
+-----+
```

Bildidee: Kleine bunte Kisten, auf jeder steht, was Codi darin gespeichert hat.

## Erklärung: Datentypen – was in die Kiste passt

Codi muss wissen, **was in die Kiste darf**:

| Datentyp            | Beispiel | Erklärung            |
|---------------------|----------|----------------------|
| <code>int</code>    | 10       | Ganze Zahlen         |
| <code>double</code> | 3.14     | Zahlen mit Komma     |
| <code>string</code> | "Hallo"  | Wörter oder Sätze    |
| <code>bool</code>   | true     | Ja/Nein, Wahr/Falsch |

---

## Erklärung: `if/else` – Entscheidungen treffen

Codi kann jetzt **Entscheidungen treffen**.

Beispiel: „Wenn du Hunger hast, hole Codi einen Snack, sonst spielt er ein Spiel.“

```
bool hunger = true;

if (hunger)
{
    Console.WriteLine("Codi holt einen Snack!");
}
else
{
    Console.WriteLine("Codi spielt ein Spiel.");
}
```

---

- `if` = wenn etwas wahr ist
- `else` = sonst

## Erklärung: Schleifen – Aufgaben wiederholen

Codi kann Aufgaben **mehrmals machen** – das nennt man **Schleife**. ↻

```
for (int i = 1; i <= 5; i++)  
{  
    Console.WriteLine("Codi hüpf! " + i);  
}
```

- `for` = Schleife, die 5 Mal läuft
- `i` = Zähler, merkt sich, wie oft Codi schon gehüpft ist

**Ausgabe:**

```
Codi hüpf! 1  
Codi hüpf! 2  
Codi hüpf! 3  
Codi hüpf! 4  
Codi hüpf! 5
```

Codi hüpf! 5

## Code: Variablen, Entscheidungen und Schleifen zusammen

```
using System;

0 references
class Program
{
    0 references
    static void Main()
    {
        string name = "Alex"; // Name merken
        int alter = 12;        // Alter merken
        bool hunger = true;    // Codi fragt: Hunger?

        Console.WriteLine("Hallo " + name + "!");

        if (hunger)
        {
            Console.WriteLine("Codi holt einen Snack!");
        }
        else
        {
            Console.WriteLine("Codi spielt ein Spiel.");
        }

        Console.WriteLine("Jetzt hüpfst Codi 3 Mal:");
        for (int i = 1; i <= 3; i++)
        {
            Console.WriteLine("Hüpf " + i);
        }
    }
}
```

## Aufgabe: Dein Codi-Abenteuer

1. Lege eine neue Variable stimmung vom Typ **string** an (z. B. "glücklich").
2. Lasse Codi je nach Stimmung etwas sagen:
  - "glücklich" → „Codi freut sich!“
  - "müde" → „Codi macht ein Nickerchen.“
3. Wiederhole eine Aufgabe 4 Mal mit einer Schleife (z. B. Codi winkt 4 Mal).

## ★★ Bonus: Zufallsgenerator für Überraschungen

Codi kann jetzt **zufällig Dinge auswählen**:

```
Random zufall = new Random();
int snack = zufall.Next(1, 4); // Zahl zwischen 1 und 3

Console.WriteLine("Codi holt Snack Nummer " + snack);
```

### Lösung:

```
using System;

0 references
class Program
{
    0 references
    static void Main()
    {
        string stimmung = "glücklich";
        if (stimmung == "glücklich")
        {
            Console.WriteLine("Codi freut sich!");
        }
        else if (stimmung == "müde")
        {
            Console.WriteLine("Codi macht ein Nickerchen.");
        }
        else
        {
            Console.WriteLine("Codi ist neutral.");
        }

        Console.WriteLine("Codi winkt 4 Mal:");
        for (int i = 1; i <= 4; i++)
        {
            Console.WriteLine("Winke " + i);
        }

        Random zufall = new Random();
        int snack = zufall.Next(1, 4);
        Console.WriteLine("Codi holt Snack Nummer " + snack);
    }
}
```

🕒 Dein Codi kann jetzt:

- Dinge merken (**Variablen**)

- Entscheidungen treffen (**if/else**)
- Aufgaben wiederholen (**Schleifen**)
- Zufällig Überraschungen erzeugen

## Kapitel 4: Methoden – eigene kleine Programme bauen

### Erklärung: Methoden – Codis Superkräfte

Stell dir vor, Codi kann bestimmte Aufgaben **immer wieder machen**, ohne dass du alles neu schreiben musst.

Diese Aufgaben nennt man **Methoden**.

- Eine Methode = **ein kleines Paket mit Befehlen**
- Du **gibst der Methode einen Namen**, und Codi führt alles darin aus, wenn du den Namen sagst

💡 Vergleich:

- Du willst, dass Codi immer dein Zimmer aufräumt:
  - Du könntest jeden Schritt immer wieder schreiben – langweilig!
  - Besser: Du packst alles in eine Methode Aufräumen().
  - Jedes Mal, wenn du Aufräumen() sagst, macht Codi alles automatisch.

---

### Visualisierung: Methoden als Schubladen

```
+-----+
| Aufräumen() | <- Eine Methode
| 1. Spielzeug weg |
| 2. Bücher sort. |
| 3. Boden wischen|
+-----+
```

Bildidee: Bunte Schublade mit Aufgaben drin – man zieht die Schublade raus und alles wird erledigt.

## Code: Eine Methode erstellen

```
using System;

0 references
class Program
{
    0 references
    static void Main()
    {
        Console.WriteLine("Codi macht sich bereit!");
        Aufräumen(); // Methode aufrufen
        Console.WriteLine("Codi ist fertig!");
    }

    1 reference
    static void Aufräumen()
    {
        Console.WriteLine("1. Spielzeug weglegen");
        Console.WriteLine("2. Bücher sortieren");
        Console.WriteLine("3. Boden wischen");
    }
}
```

- `static void Aufräumen()` → Methode definieren
- `Aufräumen();` → Methode **aufrufen**, Codi macht die Aufgaben

## Ausgabe:

```
Codi macht sich bereit!
1. Spielzeug weglegen
2. Bücher sortieren
3. Boden wischen

Codi ist fertig!
```

## Erklärung: Methoden mit Eingaben (**Parameter**)

Manchmal braucht Codi **Infos von dir**, z. B. wie viele Kekse er bringen soll. Hier kommen **Parameter** ins Spiel:

```
0 references
static void KekseBringen(int anzahl)
{
    Console.WriteLine("Codi bringt " + anzahl + " Kekse!");
}
```

**Aufruf:**

```
KekseBringen(3); // Codi bringt 3 Kekse
KekseBringen(5); // Codi bringt 5 Kekse
```

**Ausgabe:**

```
Codi bringt 3 Kekse!
Codi bringt 5 Kekse!
```

---

## Erklärung: Methoden mit Rückgabewert (**return**)

Manchmal soll Codi **etwas zurückgeben**, z. B. eine Zahl berechnen:

```
1 reference
static int Addiere(int a, int b)
{
    return a + b; // Ergebnis zurückgeben
}
```

### Aufruf:

```
int ergebnis = Addiere(5, 7);  
Console.WriteLine("5 + 7 = " + ergebnis);
```

### Ausgabe:

```
5 + 7 = 12
```

💡 Merke: **return** = Codi gibt dir das Ergebnis zurück

### Aufgabe: Codi als Superkoch

1. Erstelle eine Methode MacheSmoothie()
2. Lasse Codi sagen, welche Zutaten er hineinlegt (z. B. Banane, Apfel, Milch)
3. Optional: Füge einen Parameter anzahl hinzu, damit Codi mehrere Smoothies macht

---

### ★★ Bonus: Methoden kombinieren

Codi kann **mehrere Methoden aufrufen**:

```

0 references
static void StartTag()
{
    Aufstehen();
    FruehstueckMachen();
    LosGehts();
}

1 reference
static void Aufstehen()
{
    Console.WriteLine("Codi steht auf!");
}

1 reference
static void FruehstueckMachen()
{
    Console.WriteLine("Codi macht Frühstück!");
}

1 reference
static void LosGehts()
{
    Console.WriteLine("Codi geht zur Schule!");
}

```

Aufruf: `StartTag()`; → Codi macht alles nacheinander.

## Lösung: Smoothie-Methode

```

using System;

0 references
class Program
{
    0 references
    static void Main()
    {
        MacheSmoothie(2); // Codi macht 2 Smoothies
    }

    1 reference
    static void MacheSmoothie(int anzahl)
    {
        for (int i = 1; i <= anzahl; i++)
        {
            Console.WriteLine("Smoothie " + i + ": Banane, Apfel, Milch");
        }
    }
}

```

Ausgabe:

```

Smoothie 1: Banane, Apfel, Milch
Smoothie 2: Banane, Apfel, Milch

```

🕒 Codi kann jetzt:

- Aufgaben in **Methoden verpacken**
- Methoden mit **Eingaben** steuern
- Methoden **kombinieren**, um größere Programme zu bauen

## Kapitel 5: Objektorientierung – Klassen & Objekte (**Katze-Version**)

### Erklärung: Klassen – Codis Baupläne

Stell dir vor, Codi will **eine neue Katze bauen**.

Er kann **einen Bauplan erstellen** – darin steht, wie die Katze aussieht und was sie kann.

- Ein Bauplan = **Klasse**
- Alles, was die Katze kann = **Methoden der Klasse**
- Eigenschaften = **Attribute** (z. B. Farbe, Alter)

💡 Vergleich:

- Klasse = Backrezept
- Objekt = der Kuchen, den du daraus backst

## Visualisierung: Klasse vs. Objekt

```
Klasse: Katze
+-----+-----+
| Name    | Farbe    |
| Alter   | Größe    |
+-----+-----+
Methoden:
- Miauen()
- Rennen()

Objekt: Miezi
+-----+-----+
| Name    | Miezi    |
| Farbe   | Grau     |
| Alter   | 2 Jahre   |
+-----+-----+
Methoden: Miauen(), Rennen()
```

Bildidee: Ein Bauplan für eine Katze, daneben eine fertige Katze mit Namen und Eigenschaften.

## Code: Klasse und Objekt

```
using System;

2 references
class Katze // Das ist die Klasse
{
    public string Name; // Eigenschaft
    public string Farbe; // Eigenschaft

    1 reference
    public void Miauen() // Methode
    {
        Console.WriteLine(Name + " miaut: Miau Miau!");
    }
}

0 references
class Program
{
    0 references
    static void Main()
    {
        Katze meineKatze = new Katze(); // Objekt erstellen
        meineKatze.Name = "Miezi";
        meineKatze.Farbe = "Grau";

        meineKatze.Miauen(); // Methode aufrufen
    }
}
```

## Ausgabe:

```
Miezi miaut: Miau Miau!
```

---

## Erklärung: Konstruktoren – Objekte sofort starten

Codi kann Katzen **sofort mit Werten füttern**, damit er nicht erst alles einzeln setzen muss.

```
3 references
class Katze
{
    public string Name;
    public string Farbe;

    1 reference
    public Katze(string name, string farbe) // Konstruktor
    {
        Name = name;
        Farbe = farbe;
    }

    1 reference
    public void Miauen()
    {
        Console.WriteLine(Name + " miaut: Miau Miau!");
    }
}

0 references
class Program
{
    0 references
    static void Main()
    {
        Katze katzel = new Katze("Miezi", "Grau"); // Objekt mit Konstruktor
        katzel.Miauen();
    }
}
```

## Erklärung: Mehrere Objekte – viele Katzen

Du kannst **mehrere Katzen** aus derselben Klasse erstellen:

```
Katze katze1 = new Katze("Miezi", "Grau");  
Katze katze2 = new Katze("Luna", "Schwarz");  
  
katze1.Miauen();  
katze2.Miauen();
```

**Ausgabe:**

```
Miezi miaut: Miau Miau!  
Luna miaut: Miau Miau!
```

💡 Merke: **Klassen** = Baupläne, Objekte = die fertigen Dinge

## Aufgabe: Deine eigene Katze

1. Erstelle eine **Klasse** Katze
2. Eigenschaften: Name, Farbe
3. Methoden: Miauen(), Rennen()
4. Erstelle 2 Katzen-Objekte mit verschiedenen Namen und Farben
5. Lasse sie **miauen und rennen**

---

## ★★ Bonus: Methoden mit Parameter

Du kannst Katzen **sagen, wohin sie laufen**:

```

0 references
public void Rennen(string richtung)
{
    Console.WriteLine(Name + " rennt " + richtung);
}

```

**Aufruf:**

```

katze1.Rennen("vorwärts");
katze2.Rennen("links");

```

**Ausgabe:**

```

Miezi rennt vorwärts
Luna rennt links

```

## Lösung: Katzen-Klasse

```

5 references
class Katze
{
    public string Name;
    public string Farbe;

    2 references
    public Katze(string name, string farbe)
    {
        Name = name;
        Farbe = farbe;
    }

    2 references
    public void Miauen()
    {
        Console.WriteLine(Name + " miaut: Miau Miau!");
    }

    2 references
    public void Rennen(string richtung)
    {
        Console.WriteLine(Name + " rennt " + richtung);
    }
}

0 references
class Program
{
    0 references
    static void Main()
    {
        Katze katze1 = new Katze("Miezi", "Grau");
        Katze katze2 = new Katze("Luna", "Schwarz");

        katze1.Miauen();
        katze2.Miauen();

        katze1.Rennen("vorwärts");
        katze2.Rennen("links");
    }
}

```

Ausgabe:

```
Miezi miaut: Miau Miau!  
Luna miaut: Miau Miau!  
Miezi rennt vorwärts  
Luna rennt links
```

🔗 Codi kann jetzt:

- Klassen als Baupläne benutzen
- Katzen-Objekte erstellen
- Methoden aufrufen und Parameter übergeben
- Mehrere Objekte gleichzeitig steuern

## Kapitel 6: Mini-Projekte

### Erklärung: Jetzt wird gebaut!

Bis jetzt hast du:

- Variablen gelernt 📦
- Entscheidungen getroffen 🤖
- Schleifen genutzt ↻
- Methoden gebaut ⚙️
- Klassen verstanden 🌟

Jetzt kombinieren wir alles zu **richtigen Spielen!**

## Projekt 1: Zahlenraten-Spiel

### Erklärung

Codi denkt sich eine Zahl aus.

Du musst sie erraten!

- Zu klein → Codi sagt „größer!“
- Zu groß → Codi sagt „kleiner!“
- Richtig → Du gewinnst 🎉

### Visualisierung

Codi denkt:  → [ ? ]

Du rätst: 5 → ❌ größer!

Du rätst: 8 → ❌ kleiner!

Du rätst: 7 → ✅ RICHTIG!

---

## Code:

```
using System;

0 references
class Program
{
    0 references
    static void Main()
    {
        Random zufall = new Random();
        int geheimeZahl = zufall.Next(1, 11); // Zahl zwischen 1 und 10
        int tipp = 0;

        Console.WriteLine("Rate die Zahl zwischen 1 und 10!");

        while (tipp != geheimeZahl)
        {
            Console.Write("Dein Tipp: ");
            tipp = int.Parse(Console.ReadLine()); // Eingabe lesen

            if (tipp < geheimeZahl)
            {
                Console.WriteLine("Zu klein!");
            }
            else if (tipp > geheimeZahl)
            {
                Console.WriteLine("Zu groß!");
            }
            else
            {
                Console.WriteLine("Richtig! 🎉");
            }
        }
    }
}
```

## Aufgabe:

- Ändere den Zahlenbereich (z. B. 1–50)
- Zähle, wie viele Versuche du brauchst

---

★★ **Bonus: Füge einen Versuchszähler hinzu!**

---

## Lösung (mit Zähler):

```
using System;

0 references
class Program
{
    0 references
    static void Main()
    {
        Random zufall = new Random();
        int geheimeZahl = zufall.Next(1, 11); // Zahl zwischen 1 und 10
        int tipp = 0;

        Console.WriteLine("Rate die Zahl zwischen 1 und 10!");

        int versuche = 0;

        while (tipp != geheimeZahl)
        {
            Console.Write("Dein Tipp: ");
            tipp = int.Parse(Console.ReadLine());
            versuche++;

            if (tipp < geheimeZahl)
                Console.WriteLine("Zu klein!");
            else if (tipp > geheimeZahl)
                Console.WriteLine("Zu groß!");
            else
                Console.WriteLine("Richtig in " + versuche + " Versuchen!");
        }
    }
}
```

## Projekt 2: Quiz-Spiel

### Erklärung

Codi stellt Fragen – du antwortest!

---

### Visualisierung

Frage: Hauptstadt von Deutschland?

- 1) Berlin
- 2) Paris
- 3) Rom

Deine Antwort: 1 

---

## Code:

```
using System;

0 references
class Program
{
    0 references
    static void Main()
    {
        int punkte = 0;

        Console.WriteLine("Frage 1: Hauptstadt von Deutschland?");
        Console.WriteLine("1) Berlin 2) Paris 3) Rom");

        string antwort = Console.ReadLine();

        if (antwort == "1")
        {
            Console.WriteLine("Richtig!");
            punkte++;
        }
        else
        {
            Console.WriteLine("Falsch!");
        }

        Console.WriteLine("Du hast " + punkte + " Punkte!");
    }
}
```

---

## Aufgabe:

- Füge 2 weitere Fragen hinzu
- Gib am Ende eine Bewertung aus:
  - 3 Punkte → „Super!“
  - 1–2 Punkte → „Gut gemacht!“

★★ **Bonus: Mische Fragen zufällig mit Random**

---

Lösung:

```
using System;

0 references
class Program
{
    0 references
    static void Main()
    {
        int punkte = 0;

        Console.WriteLine("Frage 1: Hauptstadt von Deutschland?");
        Console.WriteLine("1) Berlin 2) Paris 3) Rom");
        string antwort = Console.ReadLine();

        if (antwort == "1")
        {
            Console.WriteLine("Richtig!");
            punkte++;
        }
        else
        {
            Console.WriteLine("Falsch!");
        }

        Console.WriteLine("Du hast " + punkte + " Punkte!");

        if (punkte == 3)
        {
            Console.WriteLine("Super!");
        }
        else
        {
            Console.WriteLine("Gut gemacht!");
        }
    }
}
```



## Projekt 3: Mini-Abenteuer

### Erklärung

Du baust ein kleines **Text-Abenteuer-Spiel**!

Du entscheidest, was passiert.

---

### Visualisierung:

Du stehst vor einer Höhle...

- 1) Reingehen
- 2) Weglaufen

→ Entscheidung bestimmt die Story!

---

## Code:

```
using System;

0 references
class Program
{
    0 references
    static void Main()
    {
        Console.WriteLine("Du stehst vor einer Höhle...");
        Console.WriteLine("1) Reingehen");
        Console.WriteLine("2) Weglaufen");

        string wahl = Console.ReadLine();

        if (wahl == "1")
        {
            Console.WriteLine("Du findest einen Schatz! 🏠");
        }
        else
        {
            Console.WriteLine("Du gehst sicher nach Hause.");
        }
    }
}
```

## Aufgabe:

- Füge eine zweite Entscheidung hinzu
  - Baue mindestens **3 verschiedene Enden**
- 

★★ **Bonus: Nutze Methoden für einzelne Szenen**

---

## Lösung (Idee):

```
if (wahl == "1")
{
    Console.WriteLine("Ein Drache erscheint! 🐉");
}
else
{
    Console.WriteLine("Du gehst nach Hause.");
}
```

---

## Kapitel-Zusammenfassung



Du hast jetzt:

- Dein erstes Spiel gebaut 🎮
- Eingaben vom Spieler genutzt 🌨
- Entscheidungen eingebaut 🤖
- Schleifen für Interaktion genutzt ↺

Codi ist jetzt ein echter **Game-Entwickler!** 🚀

## NÄCHSTER SCHRITT: FINALES PROJEKT

Im nächsten Kapitel bauen wir etwas richtig Cooles:



## Snake-Spiel in C#

Mit:

- Spielfeld
- Bewegung
- Essen
- Wachstum
- Game Over

---

Mega! 🚀 Jetzt kommt das **große Finale**:

## Kapitel 7: Das Snake-Spiel programmieren

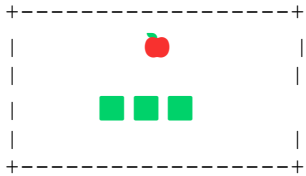
Du baust jetzt dein **eigenes Spiel** – wie ein echter Game-Entwickler! 😎  
Codi hilft dir Schritt für Schritt.

### Erklärung: Was ist Snake?

In Snake steuerst du eine Schlange:

- Sie bewegt sich über das Spielfeld
- Sie frisst Essen 🍎
- Sie wird länger
- Wenn sie sich selbst trifft → Game Over ❌

## Visualisierung:



-  = Schlange
-  = Essen

---

## Schritt 1: Spielfeld erstellen

### Erklärung

Wir benutzen die **Konsole als Spielfeld**.

### Code:

```
using System;

0 references
class Program
{
    0 references
    static void Main()
    {
        Console.CursorVisible = false; // Cursor verstecken

        int breite = 20;
        int hoehe = 10;

        for (int y = 0; y < hoehe; y++)
        {
            for (int x = 0; x < breite; x++)
            {
                Console.Write("#"); // Spielfeld
            }
            Console.WriteLine();
        }
    }
}
```

---

## Aufgabe:

Ändere die Größe des Spielfelds (z. B. 30x15)

---

## Lösung:

Einfach breite und hoehe anpassen 😊

---

## Schritt 2: Die Schlange

## Erklärung

Die Schlange ist eine Liste von Positionen.

---

## Code:

```
using System;
using System.Collections.Generic;

0 references
class Program
{
    0 references
    static void Main()
    {
        List<int x, int y> schlange = new List<int, int>();
        schlange.Add((5, 5));
        schlange.Add((4, 5));
        schlange.Add((3, 5));

        foreach (var teil in schlange)
        {
            Console.SetCursorPosition(teil.x, teil.y);
            Console.Write("O");
        }

        Console.ReadKey();
    }
}
```

## Aufgabe:

Füge ein weiteres Segment zur Schlange hinzu

---

## Schritt 3: Steuerung

### Erklärung

Jetzt kann der Spieler die Schlange steuern!

---

### Code:

```
ConsoleKey richtung = ConsoleKey.RightArrow;

if (Console.KeyAvailable)
{
    var taste = Console.ReadKey(true).Key;

    if (taste == ConsoleKey.UpArrow) richtung = ConsoleKey.UpArrow;
    if (taste == ConsoleKey.DownArrow) richtung = ConsoleKey.DownArrow;
    if (taste == ConsoleKey.LeftArrow) richtung = ConsoleKey.LeftArrow;
    if (taste == ConsoleKey.RightArrow) richtung = ConsoleKey.RightArrow;
}
```

---

## Aufgabe:

Teste alle Pfeiltasten

## Schritt 4: Essen

### Erklärung

Wir platzieren zufällig Essen auf dem Spielfeld.

---

### Code:

```
Random rand = new Random();  
(int x, int y) essen = (rand.Next(0, 20), rand.Next(0, 10));  
  
Console.SetCursorPosition(essen.x, essen.y);  
Console.Write("X");
```

---

### Aufgabe:

Ändere das Symbol für Essen

---

## Schritt 5: Wachstum

### Erklärung

Wenn die Schlange das Essen frisst → sie wächst!

---

### Code:

```
if (kopf.x == essen.x && kopf.y == essen.y)  
{  
    schlange.Add(schlange[schlange.Count - 1]); // wächst  
}
```

## Aufgabe:

Lass neues Essen erscheinen

---

## Schritt 6: Game Over

### Erklärung

Wenn die Schlange sich selbst trifft → Ende

---

### Code:

```
foreach (var teil in schlange)
{
    if (kopf.x == teil.x && kopf.y == teil.y)
    {
        Console.WriteLine("Game Over!");
    }
}
```

---

Komplettes Snake-Spiel

Jetzt kommt alles zusammen! ✨

```
using System;
using System.Collections.Generic;
using System.Threading;

class Program
{
    static void Main()
    {
        Console.CursorVisible = false;

        int breite = 30;
        int hoehe = 15;

        List<int, int> schlange = new List<int, int>();
        schlange.Add((10, 5));

        ConsoleKey richtung = ConsoleKey.RightArrow;
        Random rand = new Random();
```

```

(int x, int y) essen = (rand.Next(0, breite), rand.Next(0, hoehe));
while (true)
{
    if (Console.KeyAvailable)
    {
        var taste = Console.ReadKey(true).Key;
        richtung = taste;
    }

    var kopf = schlange[0];

    if (richtung == ConsoleKey.RightArrow) kopf.x++;
    if (richtung == ConsoleKey.LeftArrow) kopf.x--;
    if (richtung == ConsoleKey.UpArrow) kopf.y--;
    if (richtung == ConsoleKey.DownArrow) kopf.y++;

    schlange.Insert(0, kopf);
    schlange.RemoveAt(schlange.Count - 1);

    if (kopf.x == essen.x && kopf.y == essen.y)
    {
        schlange.Add(schlange[schlange.Count - 1]);
        essen = (rand.Next(0, breite), rand.Next(0, hoehe));
    }

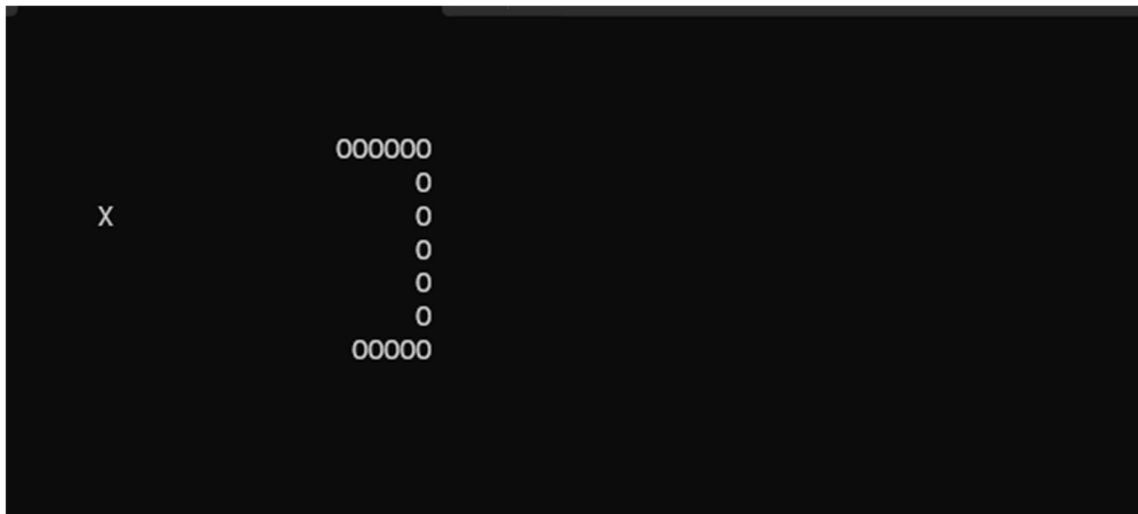
    Console.Clear();

    Console.SetCursorPosition(essen.x, essen.y);
    Console.Write("X");

    foreach (var teil in schlange)
    {
        Console.SetCursorPosition(teil.x, teil.y);
        Console.Write("O");
    }

    Thread.Sleep(150);
}
}

```



## Deine Mission

👉 Verbessere dein Spiel:

- ★ Punkte-System hinzufügen
  - ★ Geschwindigkeit erhöhen
  - ★ Wände einbauen
  - ★ Farben nutzen
- 

## Abschluss

Du hast jetzt:

- Ein echtes Spiel programmiert 
- Logik, Schleifen & Klassen genutzt
- Codi zum Game-Entwickler gemacht

👉 Du bist jetzt offiziell: **Junior C# Game Developer!** 🏆

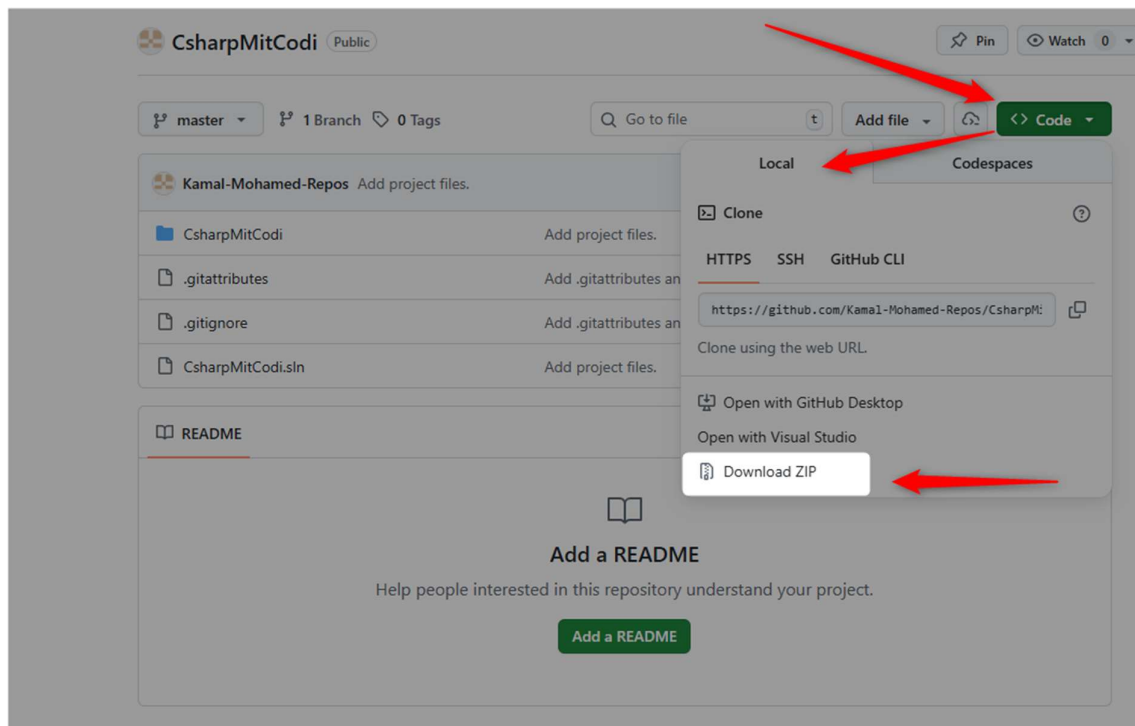
# Anhang: So startest du das C#-Programm Snake-Spiel

## 1. Code von GitHub herunterladen

1. Öffne diese Seite im Internet:

👉 <https://github.com/Kamal-Mohamed-Repos/CsharpMitCodi>

2. Klicke auf den grünen „Code“-Button.
3. Wähle „Download ZIP“ aus.
4. Speichere die Datei auf deinem Computer.
5. Entpacke die ZIP-Datei (Rechtsklick → „Alle extrahieren“).



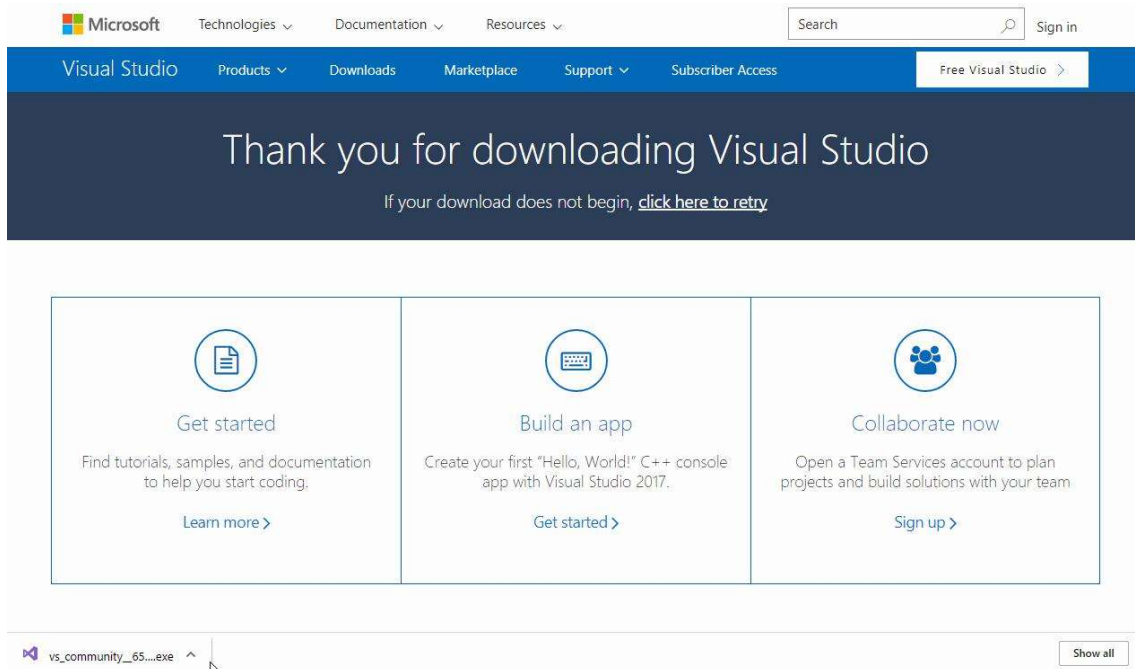
## 2. Visual Studio kostenlos installieren

1. Gehe auf die offizielle Seite von Microsoft:

👉 <https://visualstudio.microsoft.com/de/>

2. Lade **Visual Studio Community** herunter (kostenlos).
3. Starte die Installation.

4. Wichtig: Setze ein Häkchen bei  
„NET-Desktopentwicklung“
5. Klicke auf **Installieren** und warte, bis alles fertig ist.



# Visual Studio Installer

Installed

Available



## Visual Studio Community 2019 Preview

16.5.0 Preview 1.0

Powerful IDE, free for students, open-source contributors, and individuals

[Release notes](#)

Modify

Launch

More ▾



## Visual Studio Community 2019

16.4.1

Powerful IDE, free for students, open-source contributors, and individuals

[Release notes](#)

Modify

Launch

More ▾



## Visual Studio Community 2017

15.9.18

Free, fully-featured IDE for students, open-source and individual developers

[Release notes](#)

Modify

Launch

More ▾

Visual Studio Installer

Modifying — Visual Studio Enterprise 2017 — 15.8.1

**Workloads** Individual components Language packs Installation locations

Windows (3)

- ☒ .NET desktop development  
Build WPF, Windows Forms, and console applications using C#, Visual Basic, and F#.
- ☒ Universal Windows Platform development  
Create applications for the Universal Windows Platform with C#, VB, JavaScript, or optionally C++.
- ☒ Desktop development with C++  
Build Windows desktop applications using the Microsoft C++ toolset, ATL, or MFC.

Web & Cloud (7)

- ☒ ASP.NET and web development  
Build web applications using ASP.NET, ASP.NET Core, HTML/JavaScript, and Containers including Docker support.
- ☒ Azure development  
Azure SDKs, tools, and projects for developing cloud apps, creating resources, and building Containers including...
- ☒ Python development  
Editing, debugging, interactive development and source control for Python.
- ☒ Node.js development  
Build scalable network applications using Node.js, an asynchronous event-driven JavaScript runtime.
- ☐ Data storage and processing  
Connect, develop, and test data solutions with SQL Server, Azure Data Lake, or Hadoop.
- ☐ Data science and analytical applications  
Languages and tooling for creating data science applications, including Python, R and F#.
- ☒ Office/SharePoint development  
Create Office and SharePoint add-ins, SharePoint solutions, and VSTO add-ins using C#, VB, and JavaScript.

Mobile & Gaming (5)

- ☐ Mobile development with .NET  
Build cross-platform applications for iOS, Android or Windows using Xamarin.
- ☐ Game development with Unity  
Create 2D and 3D games with Unity, a powerful cross-platform development environment.

Location  
C:\Program Files (x86)\Microsoft Visual Studio\2017\Enterprise [Change...](#)

By continuing, you agree to the [license](#) for the Visual Studio edition you selected. We also offer the ability to download other software with Visual Studio. This software is licensed separately, as set out in the [3rd Party Notices](#) or in its accompanying license. By continuing, you also agree to those licenses.

Total space required -2.29 GB

[Install while downloading](#) [Modify](#)

1.17.1296.827

**Installation details**

Desktop development with C++

Included

- ☒ Visual C++ core desktop features

Optional

- ☒ Just-In-Time debugger
- ☒ VC++ 2017 version 15.8 v14.15 latest v141 tools
- ☒ C++ profiling tools
- ☒ Windows 10 SDK (10.0.17134.0)
- ☒ Visual C++ tools for CMake
- ☒ Visual C++ ATL for x86 and x64
- ☒ Test Adapter for Boost.Test
- ☒ Test Adapter for Google Test
- ☐ Windows 8.1 SDK and UCRT SDK
- ☐ Windows XP support for C++
- ☐ Visual C++ MFC for x86 and x64
- ☒ C++/CLI support
- ☐ Modules for Standard Library (experimental)
- ☐ IncrediBuild - Build Acceleration
- ☐ Windows 10 SDK (10.0.16299.0) for Desktop C++
- ☐ Windows 10 SDK (10.0.15063.0) for Desktop C++
- ☐ Windows 10 SDK (10.0.14393.0)
- ☐ Windows 10 SDK (10.0.10586.0)
- ☐ Windows 10 SDK (10.0.10240.0)
- ☐ VC++ 2015.3 v14.00 (v140) toolset for desktop

Universal Windows Platform development

Included

- ☒ Blend for Visual Studio
- ☒ .NET Native and .NET Standard
- ☒ NuGet package manager
- ☒ Universal Windows Platform tools
- ☒ Windows 10 SDK (10.0.17134.0)

Optional

- ☒ IntelliTrace
- ☐ USB Device Connectivity
- ☒ C++ Universal Windows Platform tools
- ☒ Graphics debugger and GPU profiler for DirectX
- ☐ Windows 10 Mobile Emulator (Fall Creators Upd...
- ☐ Modules for Standard Library (experimental)

### 3. Projekt öffnen und starten

1. Öffne **Visual Studio**.
2. Klicke auf  
👉 „Projekt oder Lösung öffnen“
3. Gehe in den entpackten Ordner.
4. Öffne die Datei mit der Endung **.sln**  
(das ist das Projekt).
5. Klicke oben auf den grünen **Start-Button**

